CMTAT Equivalency Assessment Criteria

Table of Contents

Document Version

`v0.2.0`

Note:

- versions with the `rc` suffix are draft versions.
- version before `1.0` are also draft versions

How to Use This Document

- Use the **CMTAT Function Equivalency Table** as the fillable assessment checklist.
- Use **Guideline for New Blockchain Implementations** as reference guidance when designing or mapping non-Solidity implementations.

General Note

- The listed functionalities are the **minimal set** required for each module.
- The key words "MUST", "MUST NOT", "REQUIRED", "SHOULD", and "MAY" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

Warning

An implementation MAY satisfy the CMTAT standard while still failing to meet the criteria required for tokenized shares under Swiss law at the underlying-ledger level. In particular, compliance with CMTAT does not, by itself, demonstrate that decentralization-related legal criteria are satisfied.

CMTAT Function Equivalency Table

Metadata

- Implementation language: *(to be filled)*
- Implementation version: *(to be filled)*

Token Attributes

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
1	Name attribute	ERC20 `name`	Public (`view`)				
2	Ticker symbol attribute	ERC20 `symbol`	Public (`view`)				
3	Reference to legally required documentation	`terms`	Public (`view`)				
4	No fractions	ERC20 `decimals`	Public (`view`)	- Decimals must be set to zero unless governing law permits fractions. - CMTAT Solidity allows configurable decimals at deployment			

For CMTAT reference implementations, decimals SHOULD be configurable rather than defaulting to zero, to support use cases beyond tokenized shares in Switzerland.

Note

> This subsection can be used to detail how mandatory token attributes are implemented and to document specific legal, business, or chain-specific cases.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
5	Token ID attribute	`tokenId`	Public (`view`)	Optional parameter.			

For CMTAT reference implementations, `tokenId` SHOULD be included.

Note

> This subsection can be used to detail optional token attributes implemented by the target system and to explain specific cases where an optional field is omitted or represented differently.

Token module

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
6	Know total supply	ERC20 `totalSupply`	Public (`view`)				
7	Know balance	ERC20 `balanceOf`	Public (`view`)				
8	Transfer tokens	ERC20 `transfer`	Token holder (`msg.sender`)				
9	Create tokens	`mint` / `batchMint`	Role-restricted (issuer/minter authorized)				
10	Cancel tokens	`burn` / `batchBurn` / `burnFrom`	Role-restricted (issuer/burner authorized)	Implementations SHOULD use a dedicated issuer/authorized burn path for forced cancellation scenarios.			

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
11	Approve	ERC20 `approve(address spender, uint256 value)`	Token holder	Grants a delegate permission to transfer a specific amount of tokens from the token account. This is optional, but implementations SHOULD include it since secondary market capability may depend on delegated approval to automate trading and settlement for regulated entities. Issuers SHOULD consult relevant trading and settlement venues if listing is contemplated.			

Note

> This subsection can be used to detail how each mandatory function is implemented, including role model, execution flow, and specific chain-level behavior.

Pause module (mandatory)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
12	Pause tokens	`pause`	Role-restricted (pauser/admin authorized)	Pause must prevent all transfers until `unpause` is called.			
13	Unpause tokens	`unpause`	Role-restricted (pauser/admin authorized)				
14	Deactivate contract	`deactivateContract`	Role-restricted (admin authorized)	Must permanently disable the token (except in upgradeability patterns where deactivation behavior is explicitly defined).			

Enforcement

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
15	Freeze	`freeze` or `setAddressFrozen(true)` *(inferred from extracted PDF text)*	Role-restricted (compliance/admin authorized)	Must block transfers to and from a given address. Single-function implementations are acceptable if they set a frozen status.			
16	Unfreeze	`unfreeze` or `setAddressFrozen(false)` *(inferred from extracted PDF text)*	Role-restricted (compliance/admin authorized)	Single-function implementations are acceptable if they clear a frozen status.			

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
17	Enforce a transfer	`forcedTransfer(address from, address to, uint256 value)`	Role-restricted (operator/compliance authorized)	Enforcement transfer is performed via `forcedTransfer`.			
18	Partial freeze	`freezePartialTokens(address account, uint256 value)` / `unfreezePartialTokens(address account, uint256 value)`	Role-restricted (operator/compliance authorized)	Intended only to block a sold amount to avoid double-spend during settlement.			

Transfer restriction (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
19	Conditional transfer request	`RuleConditionalTransferLight.detectTransferRestriction(from, to, value)` / `detectTransferRestrictionFrom(spender, from, to, value)` and `approvedCount(from, to, value)`	Public (`view`)	Request is represented by a transfer restricted until approval count is non-zero.			
20	Conditional transfer approval	`RuleConditionalTransferLight.approveTransfer(from, to, value)` (or `approveAndTransferIfAllowed`)	Role-restricted (compliance/approver authorized)	Approval is consumed on transfer via `transferred(...)`; cancellation via `cancelTransferApproval(...)`.			
21	Assign to whitelist	CMTAT Allowlist: `setAddressAllowlist(account, status)`, `batchSetAddressAllowlist(accounts, status)`, `isAllowlisted(account)`; Rules whitelist: `addAddress`, `removeAddress`, `addAddresses`, `removeAddresses`, `isAddressListed`	Role-restricted for setters; public (`view`) for checks	CMTAT Allowlist and Rules whitelist are alternative whitelist implementations.			

Note

> This subsection can be used to detail the different transfer restrictions available.

Access Control

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
22	Grant role	`grantRole(bytes32 role, address account)` (OpenZeppelin AccessControl via CMTAT/Rules modules)	Role admin (`DEFAULT_ADMIN_ROLE` or role admin)	Used for roles such as `ALLOWLIST_ROLE`, `DEBT_ROLE`, `OPERATOR_ROLE`, `COMPLIANCE_MANAGER_ROLE`.			
23	Revoke role	`revokeRole(bytes32 role, address account)`	Role admin (`DEFAULT_ADMIN_ROLE` or role admin)	AccessControl role removal.			
24	Role attribution	`hasRole(bytes32 role, address account)` / `getRoleAdmin(bytes32 role)`	Public (`view`)	In CMTAT `AccessControlModule`, `DEFAULT_ADMIN_ROLE` is treated as having all roles in `hasRole`.			

Note

> This subsection can be used to detail the concrete authorization model (roles, admins, delegates, approvers) and implementation-specific exceptions. It MAY also be relevant to explain how access control works in the implementation being approved.

Snapshot (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
25	Schedule a snapshot	`scheduleSnapshot(uint256 time)`	Role-restricted (snapshot scheduler/admin authorized)	SnapshotEngine `ISnapshotScheduler`.			
26	Reschedule a snapshot	`rescheduleSnapshot(uint256 oldTime, uint256 newTime)`	Role-restricted (snapshot scheduler/admin authorized)	`newTime` must stay between adjacent scheduled snapshots (not before previous / not after next).			
27	Unschedule a snapshot	`unscheduleLastSnapshot(uint256 time)` / `unscheduleSnapshotNotOptimized(uint256 time)`	Role-restricted (snapshot scheduler/admin authorized)	`unscheduleLastSnapshot` is restricted to the latest scheduled snapshot; `unscheduleSnapshotNotOptimized` supports generic unscheduling.			
28	Snapshot time	`getAllSnapshots()` / `getNextSnapshots()`	Public (`view`)	Returns created snapshot times and pending scheduled times.			
29	Snapshot total supply	`snapshotTotalSupply(uint256 time)`	Public (`view`)	`ISnapshotState`.			
30	Snapshot balance	`snapshotBalanceOf(uint256 time, address tokenHolder)`	Public (`view`)	`ISnapshotState` (see also `snapshotInfo`).			

Note

> This subsection can be used to detail snapshot scheduling and query behavior, including timing constraints and permission specifics.

Dividend (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
31	Distribution create parameters						
32	Distribution set eligibility						

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
33	Distribution set deposit						
34	Distribution claim deposit						
35	Distribution schedule						
36	Distribution unschedule						

Note

> This subsection can be used to detail dividend/distribution workflow specifics and jurisdiction- or product-specific handling rules.
> No direct CMTAT Solidity equivalent is currently defined for these items; they are implementation-specific. However, a prototype is available on the CMTA GitHub organization: https://github.com/CMTA/IncomeVault

Credit Events (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
37	Flag as default	`setCreditEvents(CreditEvents)` -> `creditEvents().flagDefault`	Role-restricted (issuer/compliance/admin authorized)	Managed in `ICMTATCreditEvents.CreditEvents`.			
38	Remove default flag	`setCreditEvents(CreditEvents)` with `flagDefault = false`	Role-restricted (issuer/compliance/admin authorized)	Same function as 1.29 with different value.			
39	Flag as redeemed	`setCreditEvents(CreditEvents)` -> `creditEvents().flagRedeemed`	Role-restricted (issuer/compliance/admin authorized)	Managed in `ICMTATCreditEvents.CreditEvents`.			
40	Set rating	`setCreditEvents(CreditEvents)` -> `creditEvents().rating`	Role-restricted (issuer/compliance/admin authorized)	Managed in `ICMTATCreditEvents.CreditEvents`.			

Note

> This subsection can be used to detail how credit event states are updated, governed, and audited in the implementation being approved.

Debt (optional)

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
41	Guarantor identifier	`debt().debtIdentifier.guarantor` (set via `setDebt`)	Read: public (`view`); write: role-restricted (`setDebt`)	Debt module (`ICMTATDebt.DebtIdentifier`).			
42	Debtholder representative identifier	`debt().debtIdentifier.debtHolder` (set via `setDebt`)	Read: public (`view`); write: role-restricted (`setDebt`)	Debt module (`ICMTATDebt.DebtIdentifier`).			
43	Unique identifier / hash	`tokenId()` and `terms().doc.documentHash`	Public (`view`)	`tokenId` is optional (implementations MAY omit it); document hash is in `terms` metadata.			
44	Issuance date	`debt().debtInstrument.issuanceDate` (set via `setDebt` / `setDebtInstrument`)	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`ICMTATDebt.DebtInstrument`).			

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
45	Currency of payments	`debt().debtInstrument.currency` / `debt().debtInstrument.currencyContract`	Read: public (`view`); write: role-restricted (`setDebt*`)	Supports symbol-like string and token/asset contract address.			
46	Par value	`debt().debtInstrument.parValue`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`uint256`).			
47	Minimum denomination	`debt().debtInstrument.minimumDenomination`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`uint256`).			
48	Maturity date	`debt().debtInstrument.maturityDate`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
49	Interest rate	`debt().debtInstrument.interestRate`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`uint256`).			
50	Coupon payment frequency	`debt().debtInstrument.couponPaymentFrequency`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
51	Interest schedule format: A) start date/end date/period; B) start date/end date/day of period; C) date 1/date 2/date 3	`debt().debtInstrument.interestScheduleFormat`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
52	Interest payment date: A) period; B) specific date	`debt().debtInstrument.interestPaymentDate`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
53	Day count convention	`debt().debtInstrument.dayCountConvention`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
54	Business day convention	`debt().debtInstrument.businessDayConvention`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			

Note

> This subsection can be used to detail supplementary attributes and to explain specific representation or governance choices made by the implementation being approved.

Guideline for New Blockchain Implementations

If you create a version for another blockchain, use this section to build a correspondence table between the CMTAT framework, the CMTAT Solidity version, and your implementation.

Freeze

To be compatible with ERC-3643, freeze is implemented with a single function: `setAddressFrozen(targetAddress, frozenStatus)`.

For non-EVM blockchains, implementations MAY separate this into two distinct functions:

```
freeze(address targetAddress)
unfreeze(address targetAddress)
```

Note

> This subsection can be used to detail the choice made by the implementation being approved.

CMTAT Extended

In the table below, the CMTAT framework extended features are mapped to Solidity features.

CMTAT Functionalities	CMTAT Solidity corresponding features	CMTAT Allowlist	CMTAT Light	CMTAT Debt	CMTAT Standard	Present in implementation being approved (`y/n`)	Implementation details
On-chain snapshot	`snapshotModule` and `snapshotEngine`	✔	✘	✔	✔		
Forced transfer	`forcedTransfer`	✔	✘	✔	✔		
Forced burn	`forcedBurn`	✘	✔	✘	✘		
Freeze partial token	`freezePartialTokens` / `unfreezePartialTokens`	✔	✘	✔	✔		
Integrated whitelisting/allowlisting	CMTAT Allowlist	✔	✘	✘	✘		
External whitelisting/allowlisting	CMTAT with rule whitelist	✘	✘	✔	✔		
RuleEngine / transfer hook	CMTAT with RuleEngine	✘	✘	✔	✔		
Upgradeability	CMTAT Upgradeable version	✔	✔	✔	✔		
Fee payer / gasless	CMTAT with ERC-2771 module	✔	✘	✘	✔		

Note

> This section can be used to detail supplementary features implemented beyond the mandatory baseline and specific cases in the target chain.
> For non-EVM blockchains, it MAY be relevant to explain how gasless/gas sponsorship and upgradeability work in the particular blockchain targeted.

Forced Burn and Forced Transfer

In the standard burn function, tokens from a frozen wallet MUST NOT be burnable. CMTAT offers `forcedTransfer` to force a transfer or a burn.

If `forcedTransfer` is not available, implementations MAY implement only `forcedBurn` (as in CMTAT Light). Implementations MAY also implement both. In that case, only `forcedBurn` SHOULD burn tokens, and `forcedTransfer` SHOULD NOT burn tokens.

With the CMTAT Solidity version, when `forcedTransfer` is available, `forcedBurn` is not implemented to reduce contract code size. This limitation MAY not apply to other blockchains.

Note

> This subsection can be used to detail the choice made by the implementation being approved.

Implementation Details

Functionalities	CMTAT Solidity	Access Control (CMTAT Solidity)	Note	Present in implementation being approved (`y/n`)	Access Control (implementation being approved)	Implementation details
Mint while pause	✔	Role-restricted (minter/issuer authorized)	Dedicated cross-chain mint (for example `crosschainMint`) cannot be performed while paused.			
Burn while pause	✔	Role-restricted (burner/issuer authorized)	Dedicated cross-chain burn (for example `crosschainBurn`) cannot be performed while paused.			
Self-Burn for everyone	✘	Not permitted	Token holders cannot burn their own tokens; only authorized addresses can burn.			
Self-Burn for authorized addresses	✔	Role-restricted (authorized burner)				
Standard burn on a frozen address	✘	Not permitted in standard burn path	Requires `forcedTransfer` or `forcedBurn`.			
Burn tokens with `forcedTransfer`	✔	Role-restricted (operator/compliance authorized)	See notes above.			

Self-Burn

Only the issuer and authorized addresses (not the token holder) can burn a token in CMTAT Solidity, which reflects legal requirements in several jurisdictions.

Once issued, a security can only be cancelled by its issuer, not its holder. Since the token represents the security, the same rule applies. An investor who wants to exit should transfer to the issuer, who can then cancel when legally permitted.

You MAY still add self-burn in your version if it fits your legal or business context.

Supplementary features

> This section MAY be used to document supplementary features beyond the CMTAT standard that are present in the implementation being approved.

Reference

Submodules used in this project and current checked-out versions:

Submodule	Repository	Version	Commit
CMTAT	https://github.com/CMTA/CMTAT	`v3.2.0`	`49544f4de1993008acfc9e848d0bf03bd31d8579`
SnapshotEngine	https://github.com/CMTA/SnapshotEngine	`v0.3.0-1-g19e0b56`	`19e0b569bf5823aa8cec5760f080a932a9ac940e`
RuleEngine	https://github.com/CMTA/RuleEngine	`v3.0.0-rc2-2-g9c0aa70`	`9c0aa70aae08047e4062beab0f89f92bd60252c0`
Rules	https://github.com/CMTA/Rules	`v0.3.0`	`91c21c1191e84ff938892267ec443b0d1bb9efb0`